P2580R0
Tuple protocol for C-style arrays `T[N]`

Published Proposal, 2022-04-26

This version:
https://wg21.link/P2580R0

Author:
Paolo Di Giglio

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:
Library Evolution Working Group

Abstract

I propose C-style array types of known bound, `T[N]`, should be tuple-like. My aim twofold: improve their usability in contexts where they're preferable over `std::array` and provide compile-time index checks through `std::get`. The tuple-like protocol implementation I propose for `T[N]` is designed after `std::array<T, N>`'s.

Table of Contents

§ 1. Introduction

The tuple protocol has been introduced in C++11. If `T` is a tuple-like type:

- It can be destructured into `std::tuple_size_v<T>` elements;

- Its `I`-th element has type `std::tuple_element_t<I, T>` and can be extracted through the `std::get` function template.

Since C++17, the tuple protocol interacts with the core language, which allows structured bindings to expressions of tuple-like types.

The standard already mandates the following types to be tuple-like:

- `std::ranges::subrange` (since C++20),
- `std::tuple` (since C++11),
- `std::pair` (since C++11),
- and, most relevant to this paper, `std::array` (since C++11).

In this paper, I propose the standard should make C-style arrays of known bound, `T[N]`, tuple-like too. The implementation of the tuple-like protocol (i.e. `std::tuple_size`, `std::tuple_element` and `std::get`) I propose is designed after the existing one for `std::array<T, N>`.

§ 2. Motivation

As far as their tuple-like properties are concerned, `std::array<T, N>` and `T[N]` are equivalent. Both have a fixed number of elements, `N`, which is known at compile-time; each element being of type `T` and accessible by a compile-time index.

Implementing the tuple-like protocol for C-style arrays would make them eligible to be passed as parameters to:

- `std::apply` (since C++17),
- `std::make_from_tuple` (since C++17)
- and, `std::tuple_cat` (since C++11)

> Prior to [P2165R3], the choice whether to support tuple-like types besides `std::tuple` in `std::tuple_cat` was up to the implementers.

In sections § 2.1 Automatic size deduction and § 2.2 Interacting with C APIs I outline some use cases where `T[N]` may be preferable over `std::array<T, N>`. In such cases, being able to call the above functions without the need for a temporary `std::array<T, N>` would be beneficial.

§ 2.1. Automatic size deduction

Unlike `std::array`, compilers are able to deduce the size (only) of a C-style array from the number of elements in its initializer list:

```
int c_arr[] = { 0, 1, 2 };
static_assert(sizeof(c_arr) / sizeof(c_arr[0]) == 3, "");
```

This limitation of `std::array` was noted by Alisdair Meredith in [N1479] itself and lead Zhihao Yuan to float the idea of extending the implementation of the tuple protocol to C-style arrays in [ARRAY-AS-A-TUPLE].

CTAD (since C++17) for `std::array` mitigates this problem but doesn't allow a user to specify the element type and deduce the size only. Function template `std::to_array` (since C++20) does but poses constraints on the element type (namely, it has be copy- or move-constructible and non-array).

§ 2.2. Interacting with C APIs

C (or C-like) API may force users to deal with C-style arrays:

```
// File: geometry_c_api.h
#define GEOMETRY_STATUS_OK 0

struct ReferenceFrame;
int get_origin(struct ReferenceFrame* frame, double (*pt)[3]);
```

If this paper gets accepted, client code might look like this:

```
class Point
{
public:
    explicit Point(double x, double y, double z);
    // ...
};

std::optional<Point> get_origin(ReferenceFrame& frame)
{
    double pt[3] { };
    if (get_origin(&frame, &pt) != GEOMETRY_STATUS_OK)
```

```
        return { };

    return std::make_from_tuple<Point>(pt);
}
```

§ 2.3. Compile-time bound check

A useful side benefit of the tuple protocol is the bound check performed by function template `std::get`:

```
int c_arr[42]{};

// This would not not compile because index 42 is out of bounds
//std::get<42>(c_arr) = 42;

// Not OK: this is UB and compiles
c_arr[42] = 42;
```

Making `T[N]` tuple-like would help users prevent the above class of bugs without any need for static analysis tools or sanitizers.

§ 3. Impact on the standard

This proposal is a pure library extension. It proposes changes to an existing header, `<tuple>`, but it does not require changes to any standard classes or functions.

This proposal does not require changes in the core language. It does not produce changes in the core language either. Even though the tuple protocol interferes with the core language, which provides structured-binding support for tuple-like types, the standard already defines special rules for structured bindings to C-style arrays.

This proposal does not depend on any other library extension. In section § 4.1 Proposed implementation, I propose an implementation in standard C++11.

3.1. Interaction with other papers

With this proposal, `T[N]` would satisfy exposition-only concept *`tuple-like`* introduced by Corentin Jabot with [P2165R3]. So, `std::tuple`s and `std::pair`s would be constructible from and comparable with C-style arrays:

```
int c_arr[] = { 0, 1 };
//std::tuple<int, int, int> t = c_arr; // Error: different tuple size
std::pair<int, int> p = c_arr;
p == c_arr; // Ok: evaluates to true
p < c_arr;  // Ok: evaluates to false
```

```
int c_arr[] = { 0, 1, 2 };
std::tuple<int, int, int> t = c_arr;
//std::pair<int, int> p = c_arr; // Error: different tuple size
t == c_arr; // Ok: evaluates to true
t < c_arr;  // Ok: evaluates to false
```

4. Design decisions

4.1. Proposed implementation

In the following subsections, I outline my proposed implementation of the tuple protocol for C-style arrays of known bound in standard C++11. My implementation is designed after `std::array`'s.

4.1.1. `std::tuple_size`

For the `std::tuple_size` class template, I propose the following specializations:

```
namespace std {

// (ts)
template <typename T, size_t N>
struct tuple_size<T[N]> : public integral_constant<size_t, N> { };

// (ts.c)
template <typename T, size_t N>
```

```
struct tuple_size<T const[N]> : public integral_constant<size_t, N> { };

}
```

§ *4.1.1.1. `const`-qualified specialization*

Specialization `ts.c` is required because:

- The standard already defines a `const`-qualified specialization for `std::tuple_size` (and `std::tuple_element`);
- Applying cv-qualifiers to an array type applies the qualifiers to the element type and any array type whose elements are of cv-qualified type is considered to have the same cv-qualification [CPP-REF-ARRAY].

So, by not defining `ts.c`, the following code

```
using const_array_t = int const [42];
static_assert(std::tuple_size<array_t>::value == 42, "Size OK");
```

would fail to compile because the template instanciation for `std::tuple_size` is ambiguous. In fact both the following specializations would be viable candidates:

```
namespace std {

// Already in the standard
template <class T>
struct tuple_size<const T>
    : public integral_constant<size_t, tuple_size<T>::value> { };
// With T = int[42]

// Proposed in this paper
template <typename T, size_t N>
struct tuple_size<T[N]> : public integral_constant<size_t, N> { };
// With T = int const, N = 42

}
```

§ *4.1.1.2. `volatile`-qualified specializations*

The existing specializations of `std::tuple_size` (and `std::tuple_element`) for `volatile`-qualified types were deprecated in C++20 and will be removed in C++23, as per [P1831R1]. So, there is no need to provide specializations of `std::tuple_size` (nor `std::tuple_element`) for:

- `T volatile[N]`
- `T volatile const[N]`.

§ 4.1.2. `std::tuple_element`

For the `std::tuple_element` class template, I propose the following specializations:

```
namespace std {

// (te)
template <size_t Idx, typename T, size_t N>
struct tuple_element<Idx, T[N]>
{
    static_assert(Idx < N, "Index out of bounds");
    using type = T;
};

// (te.c)
template <size_t Idx, typename T, size_t N>
struct tuple_element<Idx, T const[N]>
{
    static_assert(Idx < N, "Index out of bounds");
    using type = T const;
};

}
```

§ *4.1.2.1. cv-qualified specializations*

The reasoning behind the introduction of specialization `ts.c` in § 4.1.1 std::tuple_size also motivates the introduction of specialization `te.c` here.

As for the `volatile`-qualified specializations of `std::tuple_element`, I explain why there's no need for those in section § 4.1.1.2 volatile-qualified specializations. Please note however that, by my proposal, the following code would compile:

```
using v_array_t = int volatile[42];
using cv_array_t = int volatile const[42];

static_assert(std::is_same_v<std::tuple_element_t<0, v_array_t>, int volati
static_assert(std::tuple_size_v<v_array_t> == 42, "");

static_assert(std::is_same_v<std::tuple_element_t<0, cv_array_t>, int volat
static_assert(std::tuple_size_v<cv_array_t> == 42, "");
```

I believe this to be correct. While none of the above `static_assert`s would compile after [P1831R1] with:

```
using v_array_t = std::array<int, 42> volatile;
using cv_array_t = std::array<int, 42> volatile const;
```

They would both compile with:

```
using v_array_t = std::array<int volatile, 42>;
using cv_array_t = std::array<int volatile const, 42>;
```

§ 4.1.3. `std::get`

For the `std::get` function templates, I propose:

```
namespace std {

template <size_t Idx, typename T, size_t N>
constexpr T& get(T (&arr)[N]) noexcept {
    static_assert(Idx < N, "Index out of bounds");
    return arr[Idx];
}

template <size_t Idx, typename T, size_t N>
constexpr T&& get(T (&&arr)[N]) noexcept {
```

```
    static_assert(Idx < N, "Index out of bounds");
    return move(arr[Idx]);
}

}
```

§ 4.2. Alternative implementation

Another possible implementation for the C-style array specializations of class templates `std::tuple_size` and `std::tuple_element` in C++20 is the following (courtesy of Arthur O'Dwyer):

```
namespace std {

template <typename T, size_t N>
    requires(is_same_v<T, remove_const_t<T>>)
struct tuple_size<T[N]> : public integral_constant<size_t, N> {};

template <size_t Idx, typename T, size_t N>
    requires(is_same_v<T, remove_const_t<T>>)
struct tuple_element<Idx, T[N]> {
    static_assert(Idx < N, "Index out of bounds");
    using type = T;
};

}
```

The `requires` clause SFINAEs out the specializations of `std::tuple_size` and `std::tuple_element` for `const`-qualified array types and prevents the ambiguous-template-instantiation compilation error described in section § 4.1.1 std::tuple_size.

As for the `std::get` function templates, implementing them by means of the `requires` clause is not feasible. In fact, the following:

```
namespace std {

template <size_t Idx, typename T, size_t N>
    requires(Idx < N)
constexpr T& get(T (&arr)[N]) noexcept {
```

```
    return arr[Idx];
}

template <size_t Idx, typename T, size_t N>
    requires(Idx < N)
constexpr T&& get(T (&&arr)[N]) noexcept {
    return move(arr[Idx]);
}

}
```

may lead to an inconsistent behavior with the existing implementation for `std::array<T, N>` in unevaluated contexts:

```
std::array<int, 42> cpp_arr{};
using cpp_elem_ptr_t = decltype(&std::get<42>(cpp_arr));
// cpp_elem_ptr_t is int*

//int c_arr[42]{};
//using c_elem_ptr_t = decltype(&std::get<42>(c_arr));
// error: no matching function for call to 'get<42>(int [42])'
```

§ 5. Technical specifications

In this section, I present the changes I propose to the standard. The wording is based on [N4910].

Modify section "Header `<tuple>` synopsis `[tuple.syn]`":

```
// 22.4.6, tuple helper classes
template<class T> struct tuple_size; // not defined
template<class T> struct tuple_size<const T>;

template<class... Types> struct tuple_size<tuple<Types...>>;
```

```
template <class T, size_t N> struct tuple_size<T[N]>;
template <class T, size_t N> struct tuple_size<T const[N]>;
```

```
template<size_t I, class T> struct tuple_element; // not defined
template<size_t I, class T> struct tuple_element<I, const T>;

template<size_t I, class... Types>
  struct tuple_element<I, tuple<Types...>>;
```

```
template <size_t I, class T, size_t N>
 struct tuple_element<I, T[N]>;
template <size_t I, class T, size_t N>
  struct tuple_element<I, T const[N]>;
```

```
template<size_t I, class T>
using tuple_element_t = typename tuple_element<I, T>::type;

// 22.4.7, element access
template<size_t I, class... Types>
  constexpr tuple_element_t<I, tuple<Types...>>& get(tuple<Types...>&) noex
...
template<class T, class... Types>
  constexpr const T&& get(const tuple<Types...>&& t) noexcept;
```

```
template <size_t I, class T, size_t N>
  constexpr T& get(T (&arr)[N]) noexcept;
template <size_t I, class T, size_t N>
  constexpr T&& get(T (&&arr)[N]) noexcept;
```

Modify section "Tuple helper classes `[tuple.helper]`":

```
template<class T> struct tuple_size;
```

[1] All specializations of `tuple_size` meet the *Cpp17UnaryTypeTrait* requirements (21.3.2) with a base characteristic of `integral_constant<size_t, N>` for some `N`.

```
template<class... Types>
  struct tuple_size<tuple<Types...>> : public integral_constant<size_t, siz
```

```
template <class T, size_t N>
  struct tuple_size<T[N]> : public integral_constant<size_t, N> { };

template <class T, size_t N>
  struct tuple_size<T const[N]> : public integral_constant<size_t, N> { };
```

```
template<size_t I, class... Types>
  struct tuple_element<I, tuple<Types...>> {
    using type = TI;
  };
```

[2] *Mandates:* `I < sizeof...(Types)`.

[3] *Type:* `TI` is the type of the `I`-th element of `Types`, where indexing is zero-based.

```
template <size_t I, class T, size_t N>
  struct tuple_element<I, T[N]> {
      using type = T;
  };
```

[4] *Mandates:* `I < N`.

```
template <size_t I, class T, size_t N>
  struct tuple_element<Idx, T const[N]> {
      using type = T const;
  };
```

[5] *Mandates:* `I < N`.

Append to section "Element access `[tuple.elem]`":

```
template <size_t I, class T, size_t N>
  constexpr T& get(T (&arr)[N]) noexcept;

template <size_t I, class T, size_t N>
  constexpr T&& get(T (&&arr)[N]) noexcept;
```

[9] *Mandates:* `I < N`.

[10] *Returns:* A reference to the `I`-th element of `arr`, where indexing is zero-based.

6. Acknowledgements

I'd like to thank (sorted by `[](Dev const& lhs, Dev const& rhs) { return lhs.name < rhs.name; }`)

- Arthur O'Dwyer,
- Barry Revzin,
- Giuseppe D'Angelo,
- Jason McKesson,
- Jens Maurer,
- Lénárd Szolnoki,
- Nikolay Mihaylov,
- Zhihao Yuan

for their valuable feedbacks which made this paper possible.

References

Informative References

[ARRAY-AS-A-TUPLE]
Zhihao Yuan. *Use array as a tuple*. 2012-12-31. URL: https://blog.miator.net/post/39362111475/use-array-as-a-tuple

[CPP-REF-ARRAY]
Array declaration on cpp.reference.com. URL: https://en.cppreference.com/w/cpp/language/array

[N1479]

Alisdair Meredith. *A Proposal to Add a Fixed Size Array Wrapper to the Standard Library Technical Report*. 2003-04-23. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2003/n1479.html

[N4910]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 2022-03-17. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf

[P1831R1]

JF Bastien. *Deprecating volatile: library*. 2020-02-12. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1831r1.html

[P2165R3]

Corentin Jabot. *Compatibility between tuple, pair and tuple-like objects*. 2022-01-19. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2165r3.pdf